API Security Risk Analysis

Read-only assessment of public demo APIs

Future Interns | Cyber Security Internship | Task 3 (FUTURE_CS_03)

Prepared by	Sudip Bhalla
Report date	11 October 2026
Target 1	JSONPlaceholder - https://jsonplaceholder.typicode.com (live testing)
Target 2	ReqRes - https://reqres.in (documentation review only)
Test type	Read-only, non-intrusive, black-box API review
Tools	Insomnia 13.3.1, browser DevTools (Network tab)
Classification	Public - training exercise on demo APIs

Important note on severity

Both APIs tested are public demo services built for learning, and JSONPlaceholder returns fake data. Every severity rating in this report therefore answers one question: **how serious would this finding be if the same behavior existed in a production SaaS API holding real customer data?** In the demo setting itself, the real-world impact is negligible, and none of these items is a vulnerability in JSONPlaceholder's intended design.

1. Executive Summary

This report reviews the security posture of two public demo APIs from the point of view of a security consultant advising a SaaS company. JSONPlaceholder was tested live with a small number of read-only requests (about ten, including one safe POST). ReqRes was assessed from its published documentation only.

Bottom line: if JSONPlaceholder's behavior were found in a production product, it would need urgent attention. Anyone on the internet could list every user and read their email, home address, GPS coordinates and phone number without logging in. The API also accepts any input without checking it and exposes very little protection against abuse. On the positive side, traffic is encrypted (HTTP redirects to HTTPS) and the service sits behind a CDN.

Severity	Count	What it means for a business
High	3	Customer data could be read by anyone. Fix before launch.
Medium	3	Weakens defenses and makes abuse easier. Fix in the next release cycle.
Low	2	Gives attackers small hints. Fix during routine hardening.
Informational	1	Not a confirmed issue. Needs verification.

The ReqRes documentation review (section 7) found a sound, well-layered credential design, together with several design-level risks that a team using it should manage carefully. These are observations from documentation and are not confirmed vulnerabilities.

2. Scope, Ethics and Methodology

2.1 Scope

Target	Activity	Details
JSONPlaceholder	Live read-only testing	GET /users, GET /users/1, GET /posts, one POST /posts with test data, response header and HTTPS inspection
ReqRes	Documentation review only	Authentication model, key types, session tokens, rate limit and error handling as published in the project's documentation. No requests sent.

2.2 Rules followed

- Only public demo APIs that exist for testing and learning were used.
- Only normal, read-only requests were sent, a handful in total. No loops, bursts or flooding.
- No exploitation, bypass, brute-force or guessing of credentials was attempted.
- No login or magic-link requests were sent to ReqRes, and its public demo apps were not used.
- Rate limiting was assessed by reading response headers only.

2.3 Methodology

- Sent requests with **no Authorization header** and confirmed the Auth tab was empty.
- Reviewed status codes, response bodies and response headers in Insomnia and in browser DevTools.
- Compared individual-object requests to check for sequential IDs and visible ownership checks.
- Checked HTTPS enforcement by requesting the plain http:// address.
- Sent one POST with wrong data types, script-like text and an unexpected field, to observe validation.
- Mapped findings to the OWASP API Security Top 10 (2023 edition) and rated each one for production impact.

3. Findings at a Glance

ID	Finding	Severity if production	OWASP API Top 10 (2023)	Evidence
F1	Unauthenticated access to user list and records	**High**	API2 Broken Authentication	01, 02, 04
F2	Excessive data exposure (email, address, GPS, phone, company)	**High**	API3 Broken Object Property Level Authorization	01, 02
F3	Sequential object IDs, no visible ownership check	**High**	API1 Broken Object Level Authorization	02
F4	No pagination or response size limit on /posts	**Medium**	API4 Unrestricted Resource Consumption	04
F5	Extremely high rate limit (3,600,000)	**Medium**	API4 Unrestricted Resource Consumption	03a
F6	Key security headers missing (HSTS, CSP, X-Frame-Options)	**Medium**	API8 Security Misconfiguration	03a, 03b, 05
F7	Technology disclosure via x-powered-by: Express	**Low**	API8 Security Misconfiguration	03a, 03b
F8	Conflicting cache directives on user data	**Low**	API8 Security Misconfiguration	03a, 05
F9	No input validation: wrong types, script text and unknown fields accepted	**Medium**	API3 (mass assignment) / API8	06
I1	CORS policy not fully assessed; allow-credentials: true present	**Informational**	API8 Security Misconfiguration	05

Evidence numbers refer to the screenshot files listed in Appendix A.

4. Detailed Findings

F1 Unauthenticated access to user data — High

OWASP mapping: API2:2023 Broken Authentication (missing authentication)

What we found

GET /users and GET /users/1 returned 200 OK with complete user records although no Authorization header, token or API key was sent. The Auth tab in the client was empty. GET /posts behaved the same way.

Evidence

Screenshots 01, 02 and 04 show the request URL, the empty auth configuration and the 200 OK response with data.

Business impact

On a real SaaS product, anyone could download the whole customer directory with a single request, with no account needed. This leads to privacy-law exposure (for example GDPR or India's DPDP Act), loss of customer trust and a likely mandatory breach notification.

Recommended fix

- Require authentication (for example OAuth 2.0 or signed tokens) on every endpoint that returns user data.
- Deny by default: new endpoints should be private unless explicitly marked public.
- Add automated tests that call each endpoint without credentials and expect 401.

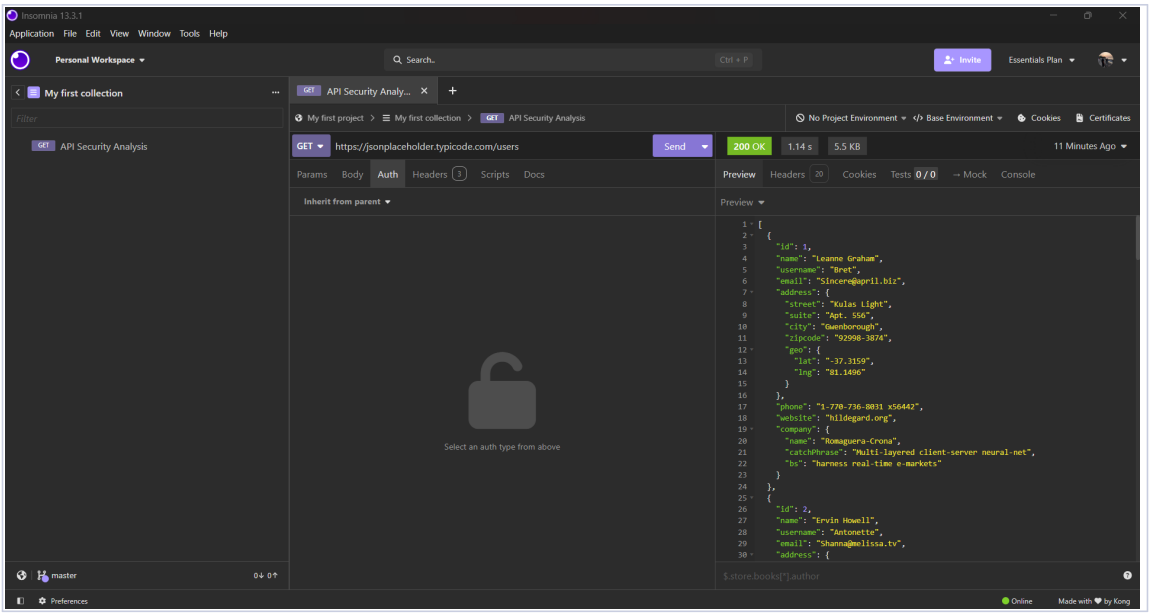


Figure 1 - GET /users returns 200 OK with no authentication configured (Evidence 01).

F2 Excessive data exposure in responses — High

OWASP mapping: API3:2023 Broken Object Property Level Authorization (excessive data exposure)

What we found

Each user object contains name, username, email, full street address and zip code, GPS coordinates (geo.lat and geo.lng), phone number, website and company details. All 10 users are returned in one 5.5 KB response.

Evidence

Screenshots 01 and 02 show the full JSON structure including the address, geo and phone fields.

Business impact

Home address plus GPS coordinates plus phone is the data set stalkers and fraudsters need. Even for an internal tool, returning more fields than the screen needs raises the damage of any single leak.

Recommended fix

- Return only the fields each client actually needs (data minimization).

- Use separate response models for public and private views instead of returning raw database records.
- Remove or restrict precise location data unless there is a clear business need.

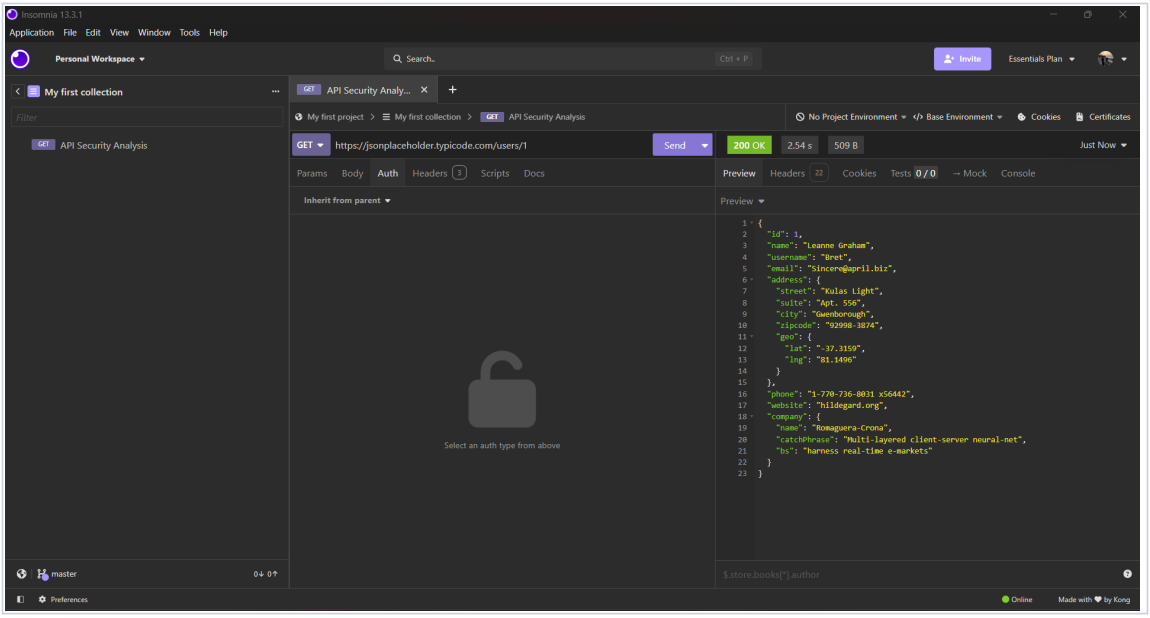


Figure 2 - GET /users/1 exposes address, GPS, phone and company in one object (Evidence 02).

F3 Sequential object IDs with no visible ownership check — High

OWASP mapping: API1:2023 Broken Object Level Authorization (BOLA)

What we found

Records are fetched by simple numeric IDs (/users/1, /users/2 and so on). No ownership or permission check is visible, because any ID returns data to an anonymous caller. Posts also carry a userId, so posts can be linked back to the user records.

Evidence

Screenshot 02 (single object by ID) and screenshot 04 (posts with userId). Only requests to documented demo endpoints were made; IDs were not enumerated.

Business impact

On a real platform this is the classic route to reading other customers' data by changing a number. BOLA is ranked number one in the OWASP API Top 10 because it is common and easy to abuse.

Recommended fix

- Check on the server that the caller is allowed to access the requested object, on every request.
- Use unpredictable identifiers (UUIDs) as an extra layer, not as the only protection.
- Include object-level access tests in the automated test suite.

F4 No pagination or response size limit — Medium

OWASP mapping: API4:2023 Unrestricted Resource Consumption

What we found

GET /posts returned the full collection (26.9 KB) in a single response, with no page size, cursor or limit parameter applied by default.

Evidence

Screenshot 04 shows 200 OK with a 26.9 KB body.

Business impact

On a large production database the same pattern lets one request pull a huge amount of data and load the servers. This raises hosting cost and makes data scraping easy.

Recommended fix

- Apply a default and a maximum page size (for example 50 records).
- Use cursor or page-based pagination for every list endpoint.
- Enforce a maximum response size at the gateway.

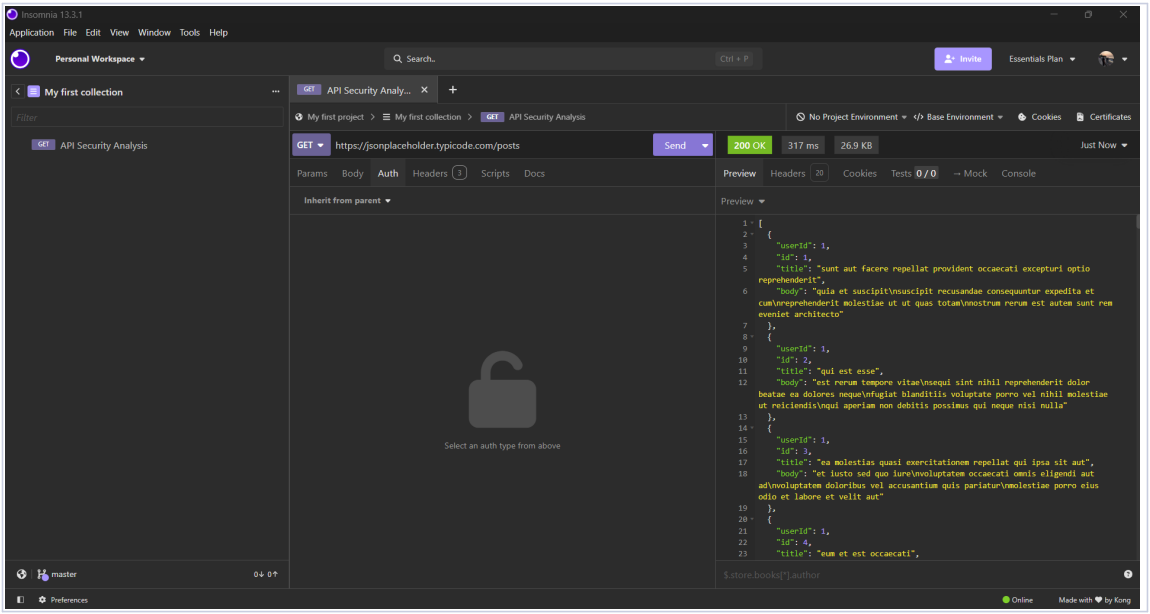


Figure 3 - GET /posts returns the full collection (26.9 KB) without pagination (Evidence 04).

F5 Very high rate limit — Medium

OWASP mapping: API4:2023 Unrestricted Resource Consumption

What we found

Rate limit headers are present, which is good, but x-ratelimit-limit is 3600000 and x-ratelimit-remaining was 3599992 after a few requests. A limit this large gives practically no protection against scraping or abuse. This may be a deliberate platform-wide setting for a free demo service.

Evidence

Screenshots 03a and 03b show the x-ratelimit-limit, x-ratelimit-remaining and x-ratelimit-reset headers. Rate limiting was assessed from headers only; no load was generated.

Business impact

In production, a limit this high would let an attacker or a faulty client send enormous numbers of requests before being slowed down, with higher infrastructure cost and exposure to credential-guessing and scraping.

Recommended fix

- Set limits based on real expected usage, per user or per API key, not only per platform.
- Use stricter limits on sensitive endpoints such as login and password reset.
- Return 429 with a Retry-After header when the limit is hit.

F6 Important security headers missing — Medium

OWASP mapping: API8:2023 Security Misconfiguration

What we found

Of the common hardening headers, only x-content-type-options: nosniff is present. Strict-Transport-Security (HSTS), Content-Security-Policy and X-Frame-Options were not returned.

Evidence

Full response header list captured in screenshots 03a, 03b and 05.

Business impact

Without HSTS, a first visit over plain HTTP can be intercepted before the redirect to HTTPS. Missing CSP and framing protection matter more when the same domain serves web pages. For a pure JSON API the practical risk is lower, but these headers are cheap defenses.

Recommended fix

- Add Strict-Transport-Security with a long max-age.
- Add a restrictive Content-Security-Policy and X-Frame-Options (or frame-ancestors).
- Review headers with an automated scanner as part of every release.

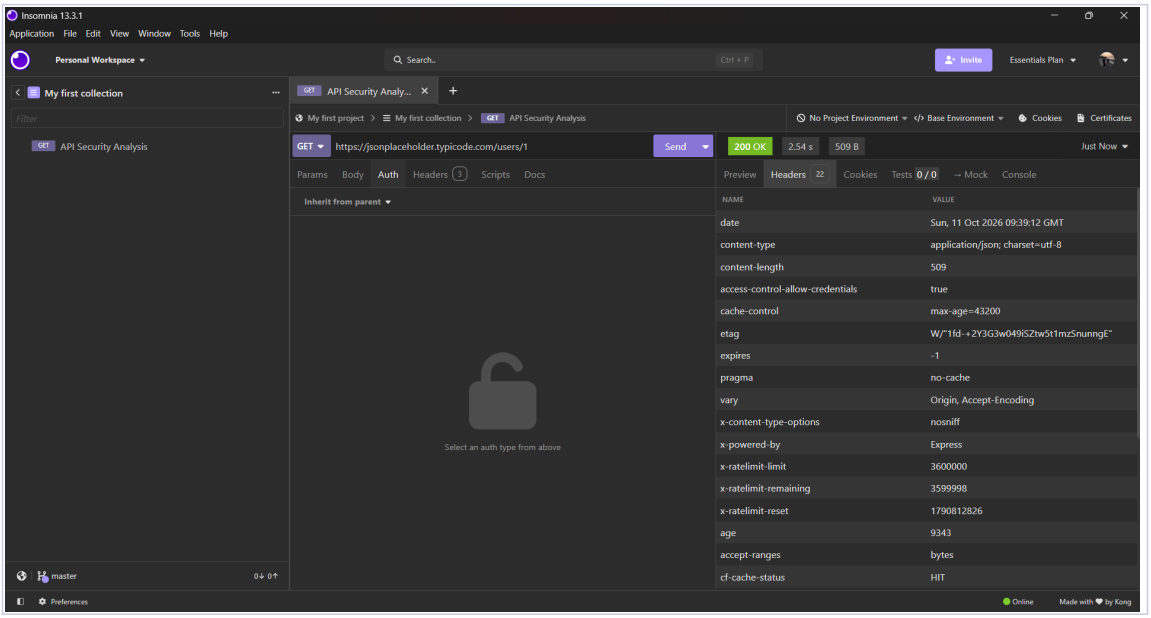


Figure 4 - Response headers: rate limit values, x-powered-by, cache directives (Evidence 03a).

F7 Technology disclosure — Low

OWASP mapping: API8:2023 Security Misconfiguration

What we found

The response header x-powered-by: Express reveals the backend framework. The server header also shows the service sits behind Cloudflare.

Evidence

Screenshots 03a, 03b and 05.

Business impact

Knowing the framework helps an attacker pick known weaknesses to try. The risk is small by itself but it is unnecessary information.

Recommended fix

- Remove the x-powered-by header (in Express: app.disable('x-powered-by') or use a hardening module).
- Keep frameworks and dependencies patched.

F8 Conflicting cache directives — Low

OWASP mapping: API8:2023 Security Misconfiguration

What we found

The same response carries cache-control: max-age=43200 (cache for 12 hours) and also pragma: no-cache and expires: -1, which say the opposite. The CDN served the response from its cache (cf-cache-status: HIT).

Evidence

Screenshots 03a, 03b and 05.

Business impact

Mixed instructions lead to unpredictable behavior. If an endpoint returned personal data, a shared cache could keep it and serve it to the wrong person or serve stale data after a change.

Recommended fix

- Choose one clear caching policy per endpoint.
- Use Cache-Control: no-store (or private) for any response containing personal data.
- Cache only public, non-personal content on the CDN.

F9 No input validation — Medium

OWASP mapping: API3:2023 (mass assignment) and API8:2023 Security Misconfiguration

What we found

One POST to /posts was sent with title as a number (12345), userId as text ("not-a-number"), HTML-like text in body (<script>test</script>) and an unexpected field (extraField). The API returned 201 Created and echoed all values back with a new id of 101. JSONPlaceholder only simulates writes and does not store the data.

Evidence

Screenshot 06 shows the request body and the 201 Created response.

Business impact

A production API that accepts and stores any type or field can end up with corrupted data, and unknown fields may overwrite protected properties (mass assignment). Script-like text that is stored and later displayed without escaping can lead to cross-site scripting. **This test did not confirm XSS**, because the text came back only as plain JSON.

Recommended fix

- Validate every request against a schema (types, lengths, formats, required and allowed fields) and reject anything else with 400 or 422.
- Use allow-lists so clients can only set the properties they should.
- Escape or sanitize output wherever stored text is displayed.

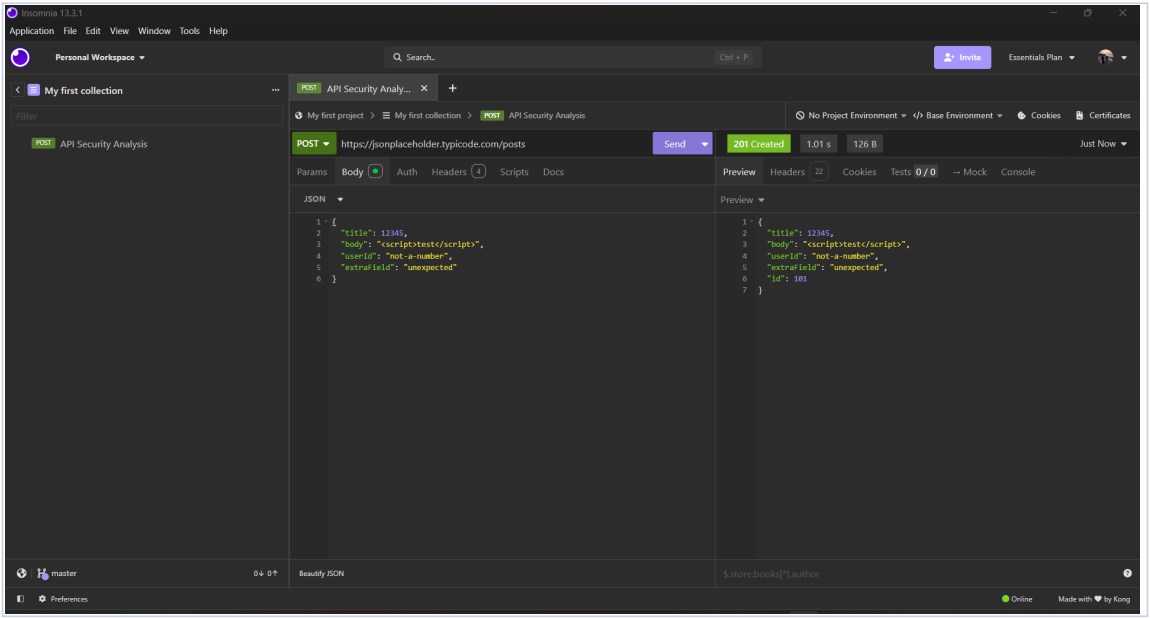


Figure 5 - POST with invalid types and an unknown field is accepted with 201 Created (Evidence 06).

I1 CORS policy not fully assessed — Informational

OWASP mapping: API8:2023 Security Misconfiguration (needs verification)

What we found

The response includes access-control-allow-credentials: true and vary: Origin, but no access-control-allow-origin header was returned for a direct browser request. Direct address-bar requests send no Origin header, so the real cross-origin behavior was not observed. No cross-origin scripts were run, to stay within scope.

Evidence

Screenshot 05 and the full header list captured from DevTools.

Business impact

Allow-credentials: true is dangerous only when combined with a wildcard or a server that reflects any origin. That has not been shown here, so this is not reported as a vulnerability.

Recommended fix

- Confirm in the API configuration that only a fixed list of trusted origins is allowed.
- Never combine allow-credentials with a wildcard or a reflected origin.

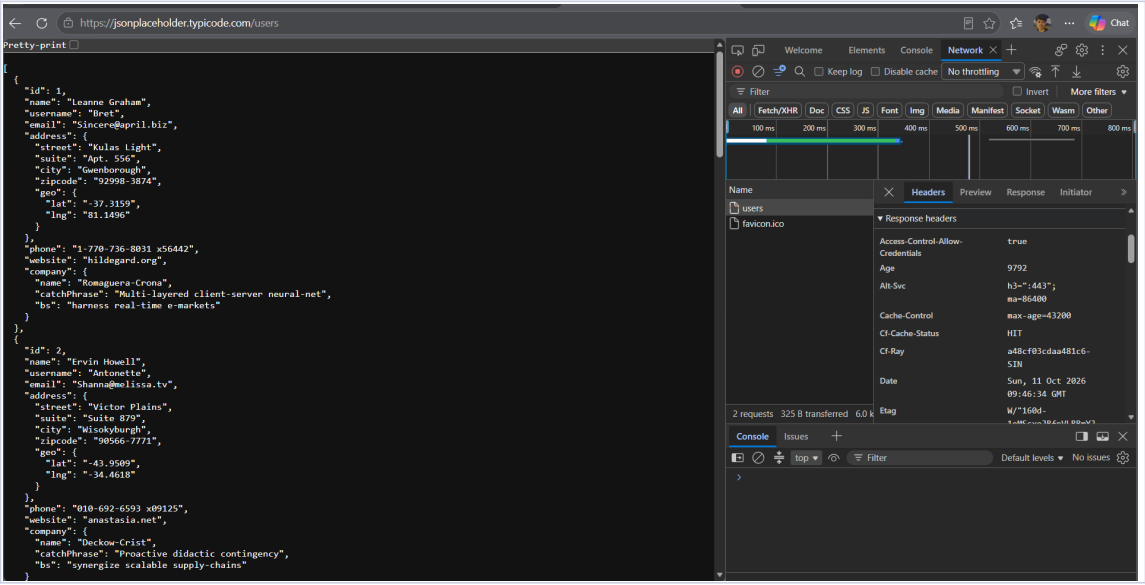


Figure 6 - DevTools response headers: allow-credentials present, allow-origin absent on direct request (Evidence 05).

5. What Is Working Well

ID	Positive control	Observation
P1	HTTPS enforced	Requesting http://jsonplaceholder.typicode.com/users redirects to https. Traffic is encrypted in transit.
P2	MIME sniffing blocked	x-content-type-options: nosniff is returned.
P3	Rate limit headers exposed	x-ratelimit-limit, remaining and reset are returned, so clients can throttle themselves (see F5 for the limit value).
P4	CDN in front of the API	Cloudflare provides caching, DDoS absorption and TLS termination.
P5	Correct content type	Responses are served as application/json; charset=utf-8.

6. Remediation Priorities

If this were a production API, the recommended order of work would be:

Priority	Action	Fixes	Effort
1 - Immediate	Add authentication to every data endpoint and deny by default	F1	Medium

Priority	Action	Fixes	Effort
1 - Immediate	Add object-level authorization checks and return only needed fields	F2, F3	Medium
2 - Next release	Schema validation on all write endpoints; pagination and sensible rate limits	F4, F5, F9	Medium
3 - Hardening	Add HSTS, CSP, X-Frame-Options; remove x-powered-by; fix cache policy; verify CORS	F6, F7, F8, I1	Low

7. ReqRes - Authentication Design Review (Documentation Only)

ReqRes now works as a backend-as-a-service: a developer creates a project and receives keys, collections and app users. This section reviews the credential model as published in the ReqRes documentation. **No requests were sent to ReqRes**, so the points below are design-level observations and not confirmed vulnerabilities.

7.1 Documented credential model

Credential	Header	Intended use	Permissions (per docs)
Private (Manage) key	x-api-key	Server or admin only	Full read/write on collections, logs and custom endpoints
Public key	x-api-key	Browser, read-only	GET records and starting the login flow
Session token	Authorization: Bearer	End users after login	Read/write on behalf of the logged-in app user

Login flow described in the docs: the app sends the user's email and project ID with the public key, the user receives a one-time code (or it is returned in test mode), and the code is exchanged for a session token.

7.2 Good design choices

- Separation of privilege: a browser-safe, read-only key is distinct from the admin key.
- Passwordless login with one-time codes means there are no stored user passwords to leak.
- Per-user session tokens support per-user data scoping.
- Rate limit headers and 429 / Retry-After handling are documented.
- The docs state clearly that the private key belongs on the server only.

7.3 Design-level risks to manage

ID	Observation	Why it matters	Severity
A1	The public key must be placed in browser code, so every visitor can read it	Anyone can call the read endpoints. Safe only if collections hold no sensitive data	Medium
A2	The private key gives full read/write access	One mistake (frontend code, public repository) exposes everything	High if leaked
A3	Public and private keys use the same header name (x-api-key)	Easy for a developer to paste the wrong key into frontend code	Medium
A4	The docs say the login code may be returned in test mode	If test mode stayed on in production, anyone could sign in as any email	High if misconfigured
A5	Session token lifetime and revocation are not stated in the sections reviewed	Long-lived tokens that cannot be revoked increase the damage of theft	Needs verification
A6	Data access depends on how each collection is configured (public read vs per-user)	A misconfigured collection could expose other users' records (BOLA)	Medium
A7	Limits on wrong login-code attempts are not described in the sections reviewed	Short one-time codes need attempt limits to resist guessing	Needs verification

A5 and A7 mean the topic was not found in the reviewed documentation, not that the protection is absent. A team adopting the service should confirm both in the full docs or with the vendor.

7.4 Recommendations for teams using this kind of service

- Keep the private key only in server-side secrets; never ship it in browser code, mobile apps or repositories.
- Rotate keys regularly and immediately after any suspected leak; run secret scanning on repositories.
- Confirm test mode is off in production and that login codes are never returned in API responses.
- Keep sensitive data out of collections that allow public key reads.
- Verify token expiry, revocation and login attempt limits before going live.

8. Conclusion and Limitations

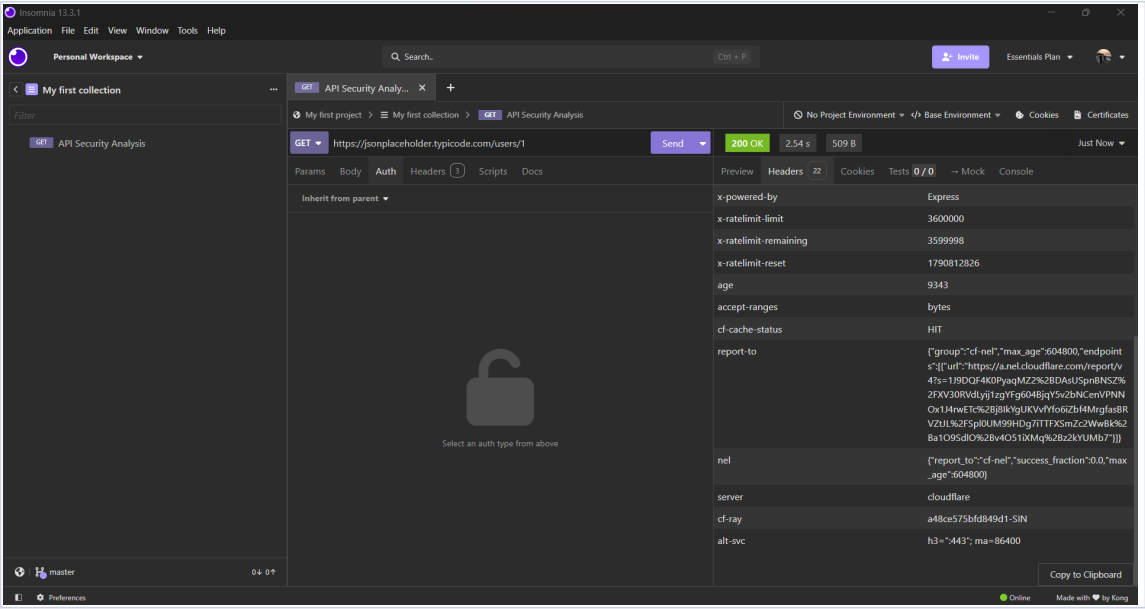
JSONPlaceholder behaves as a demo API should, open and permissive, but it is a useful teaching example of what a production API must **not** do: open endpoints, oversized responses, sequential IDs, missing validation and thin header hardening. The three High-rated items (F1 to F3) share one root cause, missing authentication and authorization, and fixing that foundation removes most of the risk.

Limitations

- Testing was limited to about ten manual read-only requests, by design.
- No authenticated testing was possible on JSONPlaceholder because it has no authentication.
- Findings F3 and F9 are based on observed behavior of a simulated API; ownership checks and persistence could not be tested.
- CORS behavior with real cross-origin requests was not tested.
- The ReqRes review covers only the documentation sections reviewed and no live testing.
- Header values and rate limits reflect the moment of testing (11 October 2026) and may change.

Appendix A - Evidence Index

File	What it shows	Used in
01_users_list_unauthenticated.png	GET /users, 200 OK, no auth, user list	F1, F2
02_user1_data_exposure.png	GET /users/1, single user object with address, GPS, phone	F1, F2, F3
03a_response_headers.png	Response headers (first part): rate limit, cache, x-powered-by	F5, F6, F7, F8
03b_response_headers.png	Response headers (last part): server, cf-ray, alt-svc	F6, F7
04_posts_unauthenticated.png	GET /posts, 26.9 KB, no pagination	F1, F4
05_cors_headers.png	Browser DevTools: allow-credentials present, allow-origin absent	I1, F6
06_input_validation.png	POST /posts with invalid data, 201 Created	F9



Appendix figure - Response headers, last part of list (Evidence 03b).

Appendix B - References

- OWASP API Security Top 10 (2023) - https://github.com/OWASP/API-Security
- API Security Checklist - https://github.com/shieldfy/API-Security-Checklist
- JSONPlaceholder - https://jsonplaceholder.typicode.com
- ReqRes documentation - https://app.reqres.in/documentation